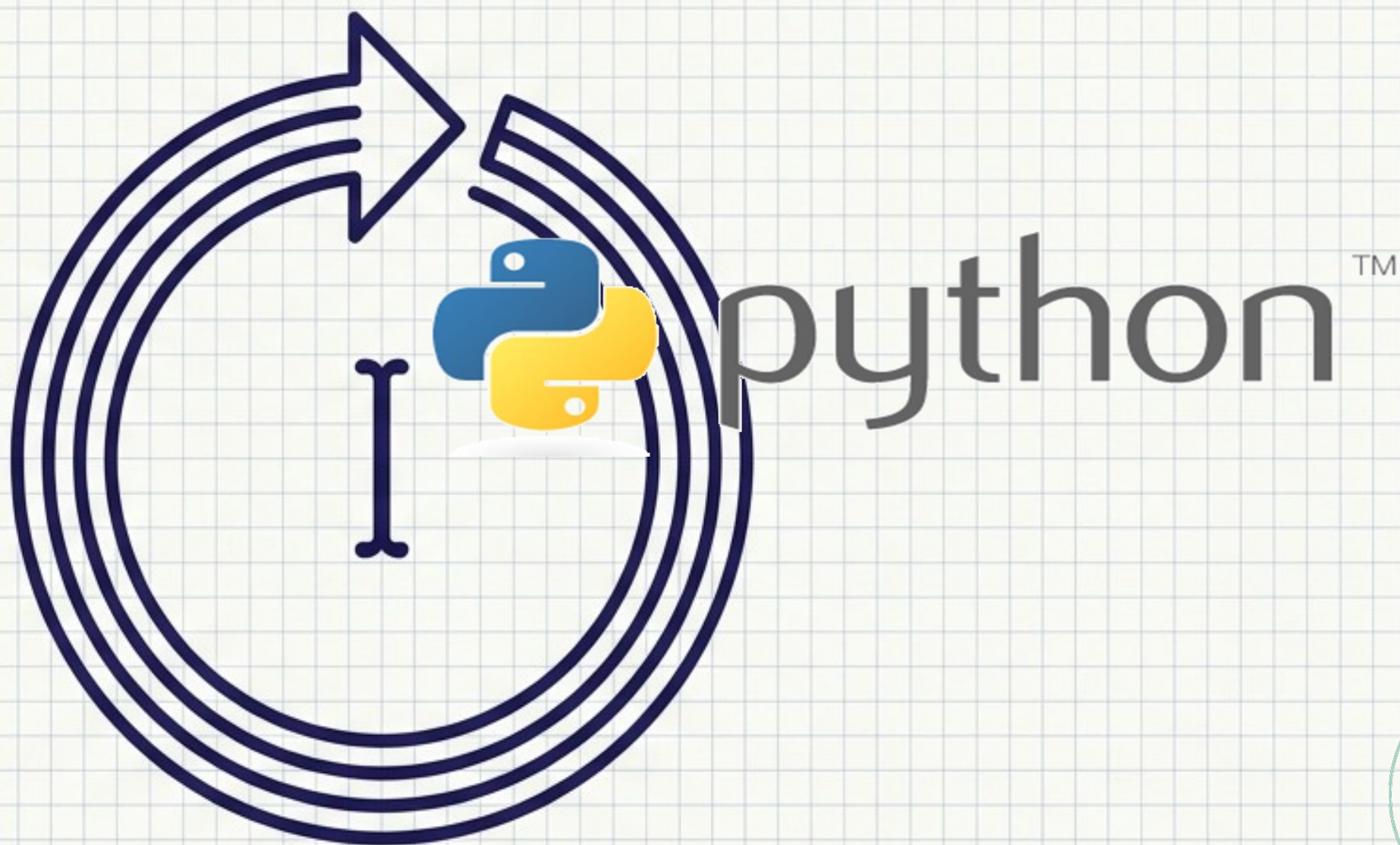


Приложение на циклични конструкции.

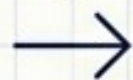
`while` цикъл в Python



КМИТ, 7 клас, Маврова-Ненкова

Еднократното изпълнение не винаги е достатъчно.

```
print("Здравей")  
print("Здравей")  
print("Здравей")  
print("Здравей")  
print("Здравей")  
print("Здравей")  
print("Здравей")  
print("Здравей")  
print("Здравей")  
...
```



The Solution



```
while условие:  
    команда
```

Циклична конструкция

- ❖ Специална конструкция от команди, която осигурява многократно изпълнение на определен код.
- ❖ Използваме цикли, за да автоматизираме повтарящи се действия, без да пишем един и същ код отново и отново.

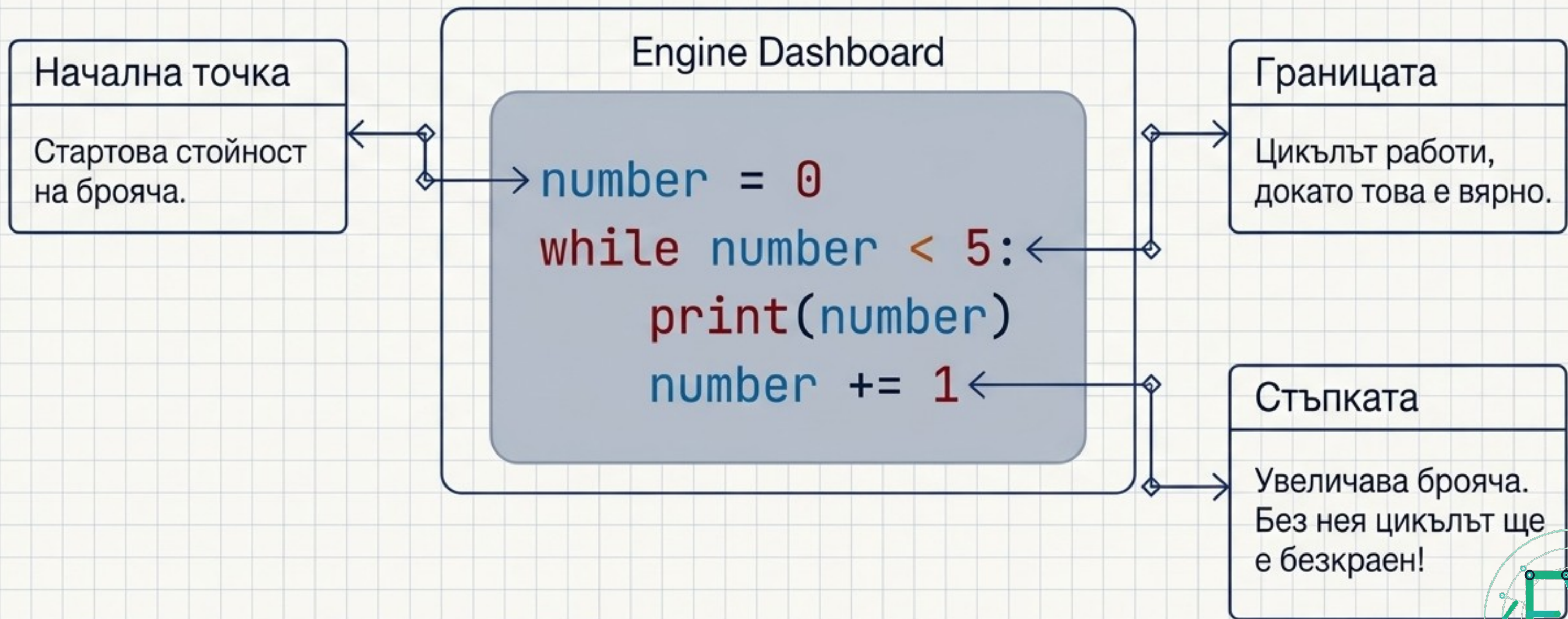

```
while условие:
```

```
    команда 1  
    команда 2
```



Докато условието е изпълнено (True), блокът от команди се повтаря.
Задължително е блокът да е с отстъп спрямо while!

Променливата играе ролята на брояч — тя контролира броя на повторенията.



Execution Trace Dashboard

Стъпка 1

- Условие: True
- Отпечатва: 0
- number става: 1



Стъпка 2

- Условие: True
- Отпечатва: 1
- number става: 2



Стъпка 3

- Условие: True
- Отпечатва: 2
- number става: 3



Стъпка 4

- Условие: True
- Отпечатва: 3
- number става: 4



Стъпка 5

- Условие: False ($5 < 5$ не е вярно!)
- Цикълът е прекъснат.

0

1

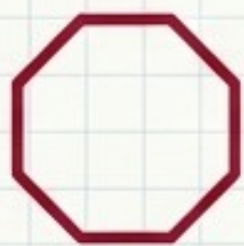
2

3

4

Сега стойността на брояча е 5.

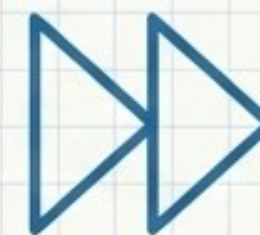
Властелините на цикъла: Специални команди



break

“Евакуация.”

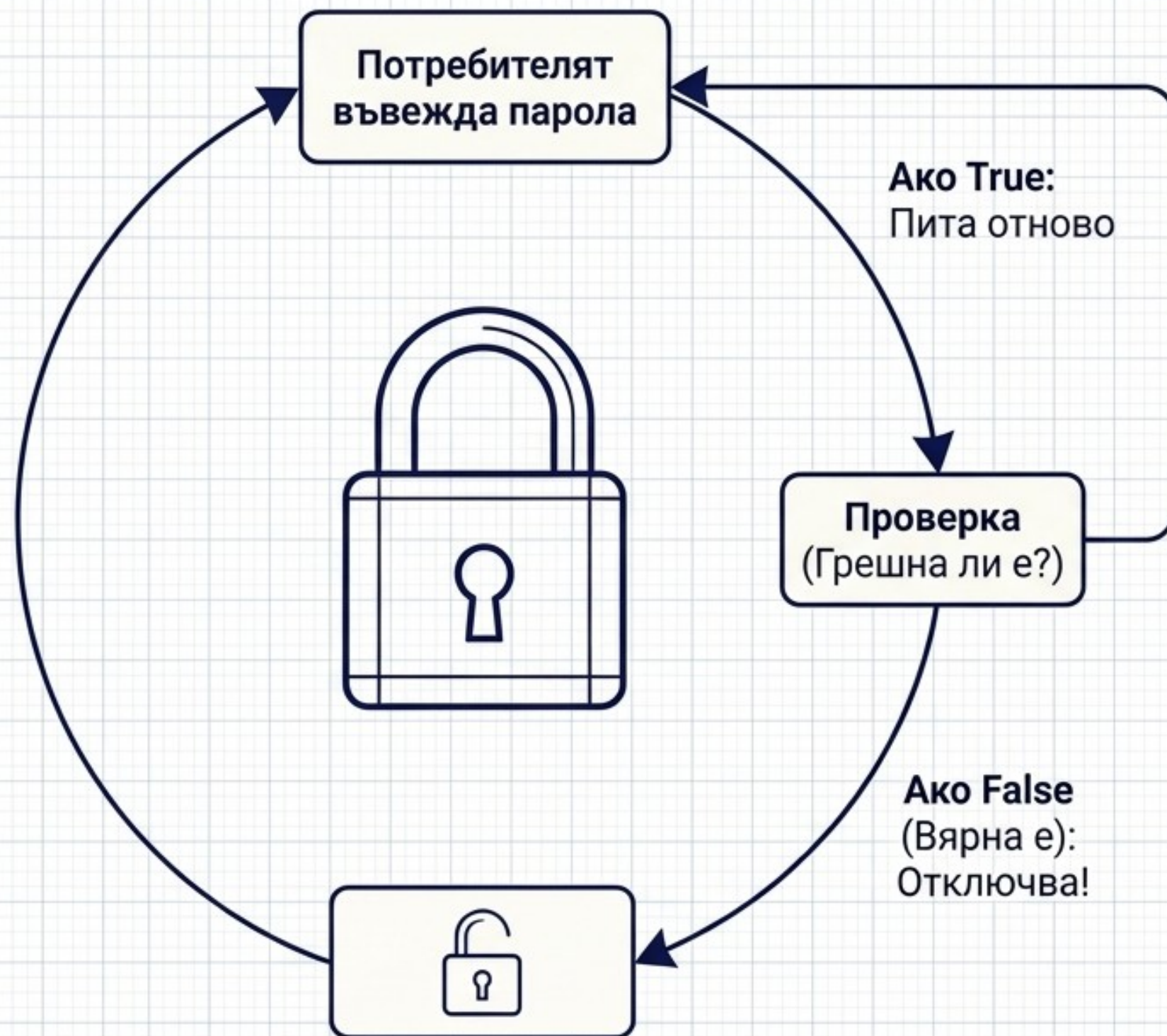
- Прекъсва целия цикъл преждевременно.
- Програмата предава контрола на следващата команда след цикъла.



continue

“Прескачане.”

- Пропуска само текущата стъпка.
- Програмата се връща директно горе при проверката на условието за следващото завъртане.

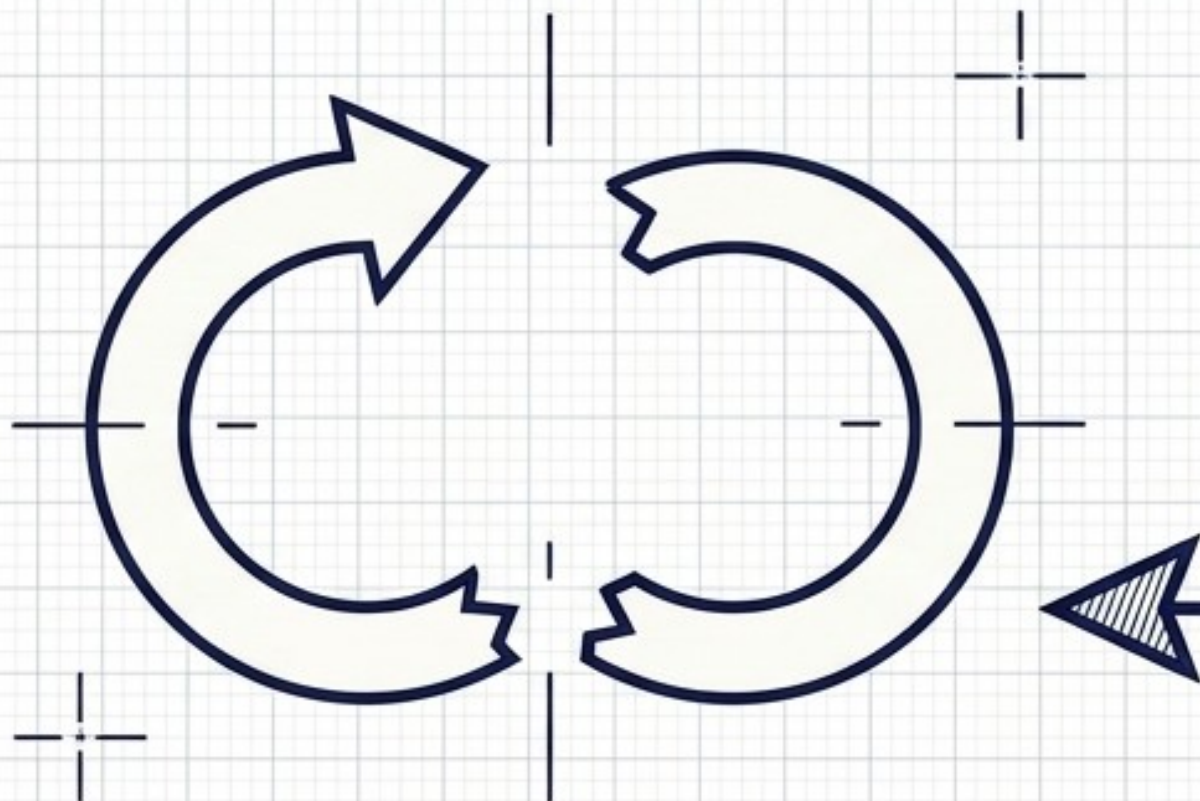


Практика: Проверка на парола

Как компютърът ни пита за парола, докато не въведем правилната?

Условието на цикъла трябва да гарантира, че процесът продължава, докато въведената парола е грешна.

Защита срещу хакери: Използване на break



- 1 **Добавяне на брояч:** Броим грешните опити. При всеки грешен опит броячът се увеличава с 1.
- 2 **Проверка за лимит:** Добавяме вътрешна проверка: дали броячът е по-голям от 5?
- 3 **Активиране на break:** Ако опитите са > 5 , отпечатваме съобщение и използваме break.

```
if greshki > 5:  
    print("Вашият акаунт ще бъде заключен!")  
    break
```


Архитектура на перфектния цикъл



Условие (Диамантът)

Задава правилото
кога цикълът
работи.



Брояч (Двигателят)

Променя се на всяка
стъпка, за да гарантира,
че цикълът няма да е
безкраен.



Контрол (Аварийните бутони)

Опционално използване
на **break** или **continue** за
управление на
извънредни ситуации.

while цикълът е интелигентна машина — тя работи автоматично,
но вие сте инженерите, които задават правилата!